

spi_master details

Reference: *ADXL362 datasheet, Rev. G (May 2023)*

0. Overview

spi_master is SPI master written in SystemVerilog, which targets the Nexys A7 100T. In this design, I only implement single-byte register read and write. This module was made to allow SPI communication between fpga and accelerometer ADXL 362 using SPI mode 0 and 5 MHz serial clock frequency. Reference to the design is *ADXL362 datasheet, Rev. G*. Till this point the RTL is complete and passes Vivado's RTL linter with 0 violations and elaborates cleanly. Each CS timing parameter is checked against Rev. G, Table 10, with the hold and disable violations found and fixed. It has not yet been simulated or tested on hardware

1. Interface

Port	Direction	Width	Description
iclk	input	1	100 MHz clock from fpga board, clocks every flip-flop in the module, including the SCLK divider
rst	input	1	active-high synchronous reset, returns the module to IDLE
start_spi	input	1	one-cycle pulse, sampled only in IDLE
rw_in	input	1	0 means read, and 1 means write
address_in	input	8	ADXL362 register address; captured at start_spi
data_in	input	8	used in write's DATA transaction
miso	input	1	data from ADXL 362 into FPGA
cs	output	1	active-low chip select to the ADXL362; low from INSTRUCTION through HOLD
mosi	output	1	data from FPGA to ADXL 362
sclk	output	1	5 MHz SPI clock derived from iclk by the divider; idles low (mode 0)

data_out	output	8	hold data from miso, updated during read's DATA state
busy	output	1	high in every state except IDLE

2. SCLK frequency

SCLK frequency use (5 MHz) due to the accelerometer stay on the same pcb with the fpga (no long wire involve, allow fast data transfer). FPGA (Nexys A7 100T) iclk is (100 MHz), while accelerometer (ADXL 362) recommended sclk frequency is (1 MHz to 8 MHz) according to Rev. G, p. 21 and Table 10 . In this case accelerometer is the limiting one. A master (Nexys A7 100T) must operate within the slave (ADXL 362) spec limit boundary, otherwise the slave won't guarantee to operate accordingly. So, the clock signal from FPGA need to step down to 5 MHz (or in between the allowed accelerometer clock frequency range). Due to short distance between components (both stay in one pcb), it is unlikely for me to deal with capacitance issue occur in long range transmission, in this case the capacitance involve is expected to be very low, which allow sharper edges . Hence, final decision sclk is 5 MHz to allow high data transmission speed between components, less than that would still allow the slave to operate correctly but in slower transmission, this value give the components 3 MHz headroom if ever the clock drifted above the intended frequency, errors are unlikely thanks to the large headroom. 5 MHz allow me to run proper clock divider. Since other frequency value like 3 MHz will force rounding, which made me miss the intended frequency, to add the division is $100 \text{ MHz} / 3 \text{ MHz} = 33.333\dots$ which will force me to use 16 count high and 17 count low which will cause me slightly off from my intended value $100 \text{ MHz} / 33 = 3.030 \text{ MHz}$ also the duty cycle is not 50%. Other than that, even we have other option to use such as 8 MHz, if ever I intended to use 8 MHz the clock divider wont produce accurate frequency value as I intended due to $100 \text{ MHz} / 8 \text{ MHz} = 12.5 \text{ cycles per period}$, which gives us 6.25 cycles per half force me to use (6 cycles low and 7 cycles high) by doing this the clock divider will give me 7.69 MHz frequency which isn't accurate to the intended value. While there is another possible value to use which dont involve rounding which is 6.25 MHz, compare to 5 MHz, 6.25MHz will give 160 ns period (80 ns high, 80 ns low) while 5 MHz will give 200 ns period (100 ns high, 100 ns low) which is easier to verify on a scope .The verdict frequency value 5 MHz make clean 10/10 split and achievable value.

3. SPI mode and clocking

Mode used in this design is SPI mode 0, this is decided after i confirmed from the datasheet's timing diagram that the ADXL 362 uses mode 0 (CPHA =0, CPOL = 0) which means the sclk idle at 0 sclk <= 0 presented during rst == 1 and cs == 1 . This is implemented in the RTL with (clk_cnt == 9 && !sclk) during rising edge the miso is captured into data_out [7 - bit_counter], while during falling edge (clk_cnt == 9 && sclk) , bit_counter increase by 1 and mosi updates the next bit, sclk still holds the old value from previous at the iclk edge, so if the old value is 1 its flip to 0 due to non blocking assignment.

iclk drives every flip-flop, with serial clock (sclk) as output to ADXL 362. To prevent glitches, which could be cause by a clock made by logic, which produce brief fake pulse and capture data at a time they shouldn't due to unintended pulse. Other than that, usage of multiple clock could cause signal passing between them crosses clock domains, which risks metastability. clock divider is used to derive sclk from iclk. In the clock divider clk_cnt is used to count every cycle of iclk (range uses was 0 to 9). This method allow the module to control sclk flips by flipping the sclk every time the clk_cnt hits 9. Which result one full sclk period (10 cycles high plus 10 low) in 20 cycle of iclk. Resulting in 5MHz sclk. During cs high or in HOLD state the clk_cnt and sclk held in 0. In the RTL,

(clk_cnt == 9 && sclk) represent falling edge, while (clk_cnt == 9 && !sclk) represent rising edge. When the module clk_cnt hits 9 and the old value of sclk is 1, the sclk goes low producing falling edge. While when the module clk_cnt hits 9 and the old value of sclk is 0, the sclk goes high producing rising edge. sclk is updated with a non-blocking assignment (<=), so on the iclk edge where clk_cnt == 9, every condition read the value sclk had before the edge and the new value only takes effect after that edge.

4. SPI commands and framing

0x0A : write register

0x0B : read register

0x0D : read FIFO (out of scope)

Address byte width : 8 bit [Rev. G register map page 25]

Register Read/Write Auto Increment : A register read or write command begins with the address specified in the command and auto-increments for each additional byte in the transfer. stop at (invalid register address: 0x3F) (Rev. G, p. 21) . (out of scope, stop after one byte of data)

a. MISO behavior

During the whole Register write transaction miso stays in high-impedance state, held by a bus keeper at its last value according to Rev. G, p. 21. This means the slave isn't driving the wire at all. While during the Register read transaction miso is held by a bus keeper at its last value until the ADDRESS state done, then the miso start feeding data_out during DATA state. Miso is never sampled during the INSTRUCTION or ADDRESS phases or during a write. Represented in RTL with :

```
else if (current_state == DATA && rw_reg == 0 && clk_cnt == 9 &&
!sclk)
begin
data_out [7 - bit_counter] <= miso;
end
```

b. Timing Flow

i. READ & WRITE (0x0A | 0x0B)

</CS down> <command byte (0x0A or 0x0B)> <address byte> <data byte> <additional data bytes for multi-byte> ... </CS up>

Note : <additional data bytes for multi-byte> not implemented

ii. Read FIFO Command (0x0D)

</CS down> <command byte (0x0D)> <data byte> <data byte> ... </CS up> (not implemented)

c. Bit order

1. instruction_reg : Bits are sent MSB-first, bit 7 when bit_counter = 0, down to bit 0 when bit_counter = 7. Represented in RTL with instruction_reg [7 - bit_counter].
2. address_reg : Bits are sent MSB-first, bit 7 when bit_counter = 0, down to bit 0 when bit_counter = 7. Represented in RTL with address_reg [7 - bit_counter].
3. data_reg : Bits are sent MSB-first, bit 7 when bit_counter = 0, down to bit 0 when bit_counter = 7. Represented in RTL with data_reg [7 - bit_counter].
4. data_out : Bits are received MSB-first, bit 7 when bit_counter = 0, down to bit 0 when bit_counter = 7. Represented in RTL with data_out [7 - bit_counter]. data_out received information from miso. Represented in RTL with data_out [7 - bit_counter] <= miso; .

5. FSM

State list

Current State (cs, bit_counter, current_state, rw_reg)	Input	Next state	Output
(0, N, 1, 0)	Falling sclk edge	(0, N+1, 1, 0)	instruction_reg[7-N]
(0, 7, 1, 0)	Falling sclk edge	(0, reset, 2, 0)	instruction_reg[0]
(0, N, 2, 0)	Falling sclk edge	(0, N+1, 2, 0)	address_reg[7-N]
(0, 7, 2, 0)	Falling sclk edge	(0, reset, 3, 0)	address_reg[0]
(0, N, 3, 0)	Falling sclk edge	(0, N+1, 3, 0)	mosi = 1 (ignored by slave) miso > data_out[7-N] on rising sclk edge
(0, 7, 3, 0)	Falling sclk edge	(0, reset(hold_cnt), 4, 0)	mosi = 1 (ignored by slave) miso > data_out[0] on rising sclk edge
(0, H(hold_cnt), 4, 0)	Rising iclk edge	(0, H+1(hold_cnt), 4, 0)	mosi = 1 (ignored by slave)
(0, 2(hold_cnt), 4, 0)	Rising iclk edge	(1, reset(hold_cnt), 5, 0)	mosi = 1 (ignored by slave)
(1, H(hold_cnt), 5, 0)	Rising iclk edge	(1, H+1(hold_cnt), 5, 0)	mosi = 1 (ignored by slave)
(1, 2(hold_cnt), 5, 0)	Rising iclk edge	(1, 0, 0, 0 or 1)	mosi = 1 (ignored by slave)
(0, N, 1, 1)	Falling sclk edge	(0, N+1, 1, 1)	instruction_reg[7-N]
(0, 7, 1, 1)	Falling sclk edge	(0, reset, 2, 1)	instruction_reg[0]
(0, N, 2, 1)	Falling sclk edge	(0, N+1, 2, 1)	address_reg[7-N]
(0, 7, 2, 1)	Falling sclk edge	(0, reset, 3, 1)	address_reg[0]
(0, N, 3, 1)	Falling sclk edge	(0, N+1, 3, 1)	data_reg[7-N]
(0, 7, 3, 1)	Falling sclk edge	(0, reset(hold_cnt), 4, 1)	data_reg[0]
(0, H(hold_cnt), 4, 1)	Rising iclk edge	(0, H+1(hold_cnt), 4, 1)	mosi = 1 (ignored by slave)
(0, 2(hold_cnt), 4, 1)	Rising iclk edge	(1, reset(hold_cnt), 5, 1)	mosi = 1 (ignored by slave)
(1, H(hold_cnt), 5, 1)	Rising iclk edge	(1, H+1(hold_cnt), 5, 1)	mosi = 1 (ignored by slave)
(1, 2(hold_cnt), 5, 1)	Rising iclk edge	(1, 0, 0, 0 or 1)	mosi = 1 (ignored by slave)
(1, 0, 0, 0 or 1)	start_spi	(0, 0, 1, rw_reg)	mosi = 1 (ignored by

Current State (cs, bit_counter, current_state, rw_reg)	Input	Next state	Output
			slave)

Legend

Falling SCLK edge : In RTL : (clk_cnt == 9 && sclk)

Rising SCLK edge : In RTL : (clk_cnt == 9 && !sclk)

bit_counter : $0 \leq N < 7$, N is bit_counter

hold_cnt : $0 \leq H < 2$, H is hold_cnt

rw_reg : 0 = Read, 1 = Write

Current_state : 0 = IDLE, 1 = INSTRUCTION, 2 = ADDRESS, 3 = DATA, 4 = HOLD, 5 = DISABLE

busy : high every state except IDLE

6. CS timing

Parameter	Definition	Datasheet min	My design
tCSS	Minimum time from CS transition (high to low) to the first sclk rising edge	100 ns	100 ns (zero margin)
tCSH	Minimum time from the last sclk falling edge to CS transition from low to high	20 ns	30 ns
tCSD	minimum time CS stays high, from CS rising edge to CS falling edge .	20 ns	≥ 40 ns

Rev. G, Table 10, Figures 41–42.

CS timing RTL

```
tCSD:
assign cs = (current_state == IDLE || current_state == DISABLE) ?
1:0;
tCSS & tCSH:
else if (cs == 1 || current_state == HOLD)
    begin
        clk_cnt <= 0;
        sclk <= 0;
    end
else if (clk_cnt == 9)
    begin
        sclk <=~ sclk;
        clk_cnt <= 0;
    end
tCSH & tCSD:
HOLD      : next_state = (hold_cnt == 2) ? DISABLE : HOLD;
DISABLE   : next_state = (hold_cnt == 2) ? IDLE : DISABLE;

always_ff @(posedge iclk)
begin
    if (rst)
        begin
            hold_cnt <= 0;
        end
    else if (hold_cnt == 2)
        begin
            hold_cnt <= 0;
        end
    else if (current_state != HOLD && current_state != DISABLE)
        begin
            hold_cnt <= 0;
        end
    else
        begin
            hold_cnt <= hold_cnt +1;
        end
end
```

Why HOLD and DISABLE exist

During the first design, after the DATA state finish the state machine transition into IDLE directly. Which this violated tCSD and tCSH, which require me to add 2 more state to satisfy the parameter based on datasheet. New states is HOLD and DISABLE. During HOLD state cs still low same from DATA phase, the sclk stop cycle after the last falling edge at the end of DATA state. The HOLD state last for 30 ns (3 iclk cycle). This is to satisfy the tCSH, the hold time that require minimum 20 ns. While during DISABLE state cs goes high create cs rising edge at the end of HOLD state and sclk stay low. The DISABLE state last for 30 ns (3 iclk cycle). The cs stay high at the end of DISABLE and keep high during IDLE this make the cs stay high at least 40 ns since IDLE adds a minimum of one cycle. This is to satisfy the tCSD, the disable time that require minimum 20 ns. This teach me to check the datasheet more detail, because the RTL might seems fine in the timing diagrams and the testbench would not have caught this on its own but might not operate correctly on the hardware.

7. Interface contract

a. Input capture

during iclk rising edge while in IDLE and with start_spi high: rw_reg capture rw_in, address_reg capture address_in, data_reg capture data_in and if rw_in is 0 instruction_reg gets 8'h0B, while if rw_in is 1 instruction_reg gets 8'h0A. Without capture, an input change halfway through ADDRESS phase, would send corrupted input to ADXL 362. As example halfway through 0x08 input during ADDRESS it changes to 0x12, this will cause the address to ADXL 362 reads PARTID (0x02), which returns 0xF2, which not what intended. This apply to all input. With capture the module only use what it sample (rw_reg, address_reg, data_reg), this will cause any changes on _in mid transaction will cause no effect on the current transaction.

b. start_spi and busy

start_spi pulse for one iclk cycle, at which each pulse only produce one transaction. Otherwise if start_spi is level the transaction will loop and produce multiple transaction, which is not as intended. Timing stays safe even with level start_spi, due to DISABLE and IDLE state provide at least 40 ns CS high time, tCSD. Busy will stay high at every state but IDLE, allow start_spi to pulse after the busy goes low, if this isnt done the start_spi will be ignored and lost because on IDLE reads start-spi, and nothing stores a pulse that arrives while busy.

8. Output behavior

mosi drive 1 during IDLE, HOLD, DISABLE and read's DATA phase. During IDLE and DISABLE, cs goes high cause the ADXL 362 to not listen to mosi. In the HOLD state there is no sclk, so the slave can't sample mosi. While during read's DATA phase mosi drives 1 while ADXL 362 drives miso with the contents of the register at the address sent during ADDRESS state. Slave ignores mosi during sending data. bit_counter selects the bit position [7 - bit_counter], so each bit is captured by the module into data_out[7-N] on the rising sclk edge. When rst is asserted data_out become 0xFF. data_out change during read's DATA phase, each bit sample on sclk rising edge. During write's DATA phase data_out holds the last byte read because during write transaction the module dont change the data_out, since data_out only change during read transaction so the game logic can do a write between reads without losing the last value it read.